Project 2

Griductive Solver

1 Introduction

Griductive[1] is a grid-deduction game in which every person is secretly a criminal or an innocent. The visible statements are true, and the player identifies people one provable verdict at a time. A correct verdict flips a card and reveals a new statement. The central rule is that the player must **NOT** guess: a verdict should be accepted only when it follows logically from the information currently available.



Hình 1: A screenshot of a typical Griductive gameplay. You can try the game via this link.

In this project, students will develop a no-guess Griductive solver using propositional logic, CNF encoding, and a DPLL SAT solver. The solver derives only logically entailed verdicts, reveals clues progressively, and detects ambiguous or inconsistent states.

[1] Griductive: play page and official how-to-play guide.

2 Gameplay

The following rules are adapted from the original Griductive version.

2.1 Board and clues

A puzzle is an $N \times N$ grid, typically with $N \in \{3, 4, 5\}$. Each cell contains one character whose hidden status is either **Criminal** or **Innocent**, together with one structured clue.

Rows are numbered from top to bottom and columns are lettered from left to right; for example, C2 denotes column C, row 2. Characters are ordered alphabetically by display name.

A few cards are face-up initially; their statuses and clues form the public knowledge base state. All other cards remain face-down. Every revealed clue is true regardless of its owner's status, so this is not a truth-teller/liar puzzle. The names and occupations carry no logical meaning unless a clue explicitly refers to them.

The required clue regions are rows, columns, the neighbors of a cell, and explicit lists of cells. A row contains all cells with the same row number, while a column contains all cells with the same column letter. The neighbors of a cell are the cells that touch it horizontally, vertically, or diagonally; the cell itself is excluded. Thus, an interior cell has eight neighbors, while edge and corner cells have fewer. An explicit region is a list of distinct cell identifiers specified by the clue.

Clues are supplied in a structured format; the GUI may display natural-language versions, but no natural-language processing is required.

2.2 Deduction loop

At step t, the knowledge base KB_t contains only revealed clues and previously proved statuses:

1. Select an unsolved character whose status is logically forced by KB_t. If several are forced, choose deterministically, such as in row-major order.
2. Submit `CRIMINAL` or `INNOCENT`. If the verdict is entailed, reveal the card and add its status and clue to form KB_{t+1}.
3. If neither status is entailed, return `UNKNOWN`; do not guess or reveal the card.
4. A rejected verdict leaves the game state unchanged. The interface reports `NOT_PROVABLE` when neither status is forced and `CONTRADICTED` when the opposite status is forced.

A valid puzzle has exactly one complete assignment of Criminal and Innocent statuses.

3 Logical Formulation

3.1 Propositional variables

For every character i, define one primary Boolean variable:

$$C_i = \begin{cases} \text{True}, & \text{if character } i \text{ is a Criminal,} \\ \text{False}, & \text{if character } i \text{ is Innocent.} \end{cases}$$

The variable map must be deterministic. Auxiliary variables may be introduced by an encoding, but they must be kept separate from the primary variables C_i.

3.2 Core clue language

The core implementation must support the following six clue templates. A *region* may be a row, a column, the neighbors of a cell, or an explicit list of cells.

Clue type	Meaning	Logical content
`FACT`	A named person has a given status.	A unit literal C_i or $\neg C_i$.
`SAME`	Two named people have the same status.	$C_i \leftrightarrow C_j$.
`DIFFERENT`	Two named people have different statuses.	$C_i \oplus C_j$.
`EXACTLY`	Exactly k people in a region are Criminals.	$\sum_{i \in R} C_i = k$.
`AT_LEAST`	At least k people in a region are Criminals.	$\sum_{i \in R} C_i \geq k$.
`AT_MOST`	At most k people in a region are Criminals.	$\sum_{i \in R} C_i \leq k$.

Bảng 1: Required structured clue templates.

For counting clues, k is an integer satisfying $0 \leq k \leq |R|$. The word `NO` may be rendered as `EXACTLY(0, region)`.

In addition to the six core templates, each team must implement at least **two distinct extensions**. An extension may be a parity constraint, a comparison between regional counts, a connectivity constraint, a compound clue, or an advanced region expression such as intersections, common neighbors, boundary cells, corners, or cells between two named characters.

3.3 CNF knowledge base

Your program must not contain a hand-written CNF formula for each test puzzle. Every active clue must be converted automatically to CNF. Known verdicts are represented as unit clauses.

Students may choose any correct cardinality encoding, but the report must derive examples. Because the required regions are small, a direct combinatorial encoding is acceptable in this case. The implementation should report the number of primary variables, auxiliary variables, and clauses generated for each puzzle.

3.4 Model versus logical consequence

A satisfying model is only one possible assignment. A verdict is valid only when it is true in *every* model of the current knowledge base. For an unsolved character i at deduction step t:

$$KB_t \models C_i \iff KB_t \wedge \neg C_i \text{ is UNSAT},$$
$$KB_t \models \neg C_i \iff KB_t \wedge C_i \text{ is UNSAT}.$$

Therefore:

- return `CRIMINAL` if $KB_t \wedge \neg C_i$ is UNSAT;
- return `INNOCENT` if $KB_t \wedge C_i$ is UNSAT;
- return `UNKNOWN` if both formulas are satisfiable; and
- return `INCONSISTENT` if KB_t itself is unsatisfiable.

Example. Consider two characters A and B with the knowledge base

$$KB = (C_A \vee C_B) \wedge (\neg C_A \vee C_B).$$

The assignment $C_A =$ False and $C_B =$ True satisfies both above clauses, so that KB is SAT. To determine the status of B, temporarily assume that B is Innocent. Then:

$$KB \wedge \neg C_B \implies C_A \wedge \neg C_A,$$

which is UNSAT. Therefore, $KB \models C_B$, and B must be Criminal. In contrast, both $KB \wedge C_A$ and $KB \wedge \neg C_A$ are satisfiable, so the status of A remains `UNKNOWN`.

4 Requirements

Before we get started, here are some important things to consider:

- The source code must be written in **Python**.
- You may use supporting libraries (e.g., for GUI or data structures), but you must implement the main algorithms **yourself**.
- There is **no restriction** on how you organize your code, but it should be professionally and clearly structured.

4.1 Reimplement the Game (10 pts)

Implement a basic, playable version of the Griductive game. The program **MUST HAVE** a GUI (a web or desktop application is acceptable). The interface must:

- Display the grid, row/column coordinates, character names and professions, known statuses, face-up clues, and face-down cards.
- Allow a user to call a character `CRIMINAL` or `INNOCENT`.
- When a verdict is proved and correct, accept the verdict, visibly mark the character's status, and flip the card to reveal that character's clue.
- Reject a verdict that is not justified by the current public knowledge without revealing the hidden clue. Distinguish:
  - `NOT_PROVABLE`: neither verdict is currently forced; and
  - `CONTRADICTED`: the opposite verdict is currently forced.
- Allow a revealed clue to be selected and visually highlight the characters/cells referenced or counted by that clue.
- Provide Load, Restart, Hint, and Auto Solve controls. Hint must identify a relevant revealed clue and/or a character whose verdict can be proved next.

The architecture must separate two roles:

- **Game Engine** owns the complete puzzle, hidden solution, and unrevealed clues.
- **Logic Agent** receives only the public knowledge state: revealed clues and proved verdicts. It must not inspect hidden labels or unrevealed clue content.

4.2 Automatic Logical Representation and CNF Encoding (10 pts)

Implement a reusable encoder that:

- Creates a deterministic mapping between characters and propositional variables.
- Validates clue parameters and referenced regions.
- Converts all clue templates to CNF.
- Adds known verdicts as unit clauses.
- Builds the knowledge base at deduction step t as

$$KB_t = \text{CNF}(\text{revealed clues}) \wedge \text{unit clauses for proved verdicts.}$$

  Hidden statuses and unrevealed clues must not be included in KB_t.

- Reports the generated variable and clause counts.

At least one direct semantic evaluator must be implemented for each required clue template. Given a complete assignment, this evaluator states whether the clue is true without calling the CNF encoder.

4.3 DPLL SAT Solver (10 pts)

Implement a SAT solver based on the **Davis-Putnam-Logemann-Loveland** (DPLL) procedure. The minimum requirements are:

1. Unit propagation.
2. Conflict detection.
3. Deterministic variable selection.
4. Recursive branching and backtracking.
5. Returning `SAT` with a complete assignment or `UNSAT`.
6. Recording decisions, propagations, backtracks, and runtime.

Pure-literal elimination and more advanced branching heuristics are optional. The SAT solver may accept assumptions directly or may solve KB_t after temporary unit clauses are added.

4.4 Deductive Logic Agent (20 pts)

Build an automatic agent on top of the DPLL solver. The agent must:

- Classify every unresolved character as `CRIMINAL`, `INNOCENT`, or `UNKNOWN`.
- Choose only a forced verdict and NEVER guess.
- Request the corresponding clue from the GameEngine after an accepted verdict.
- Update the knowledge base and continue until the puzzle is solved or another terminal state is reached.
- Produce a deduction trace containing the step number, active clue identifiers, SAT queries, verdict, and newly revealed clue.
- Provide a separate uniqueness check for the complete clue set.

The Hint function may identify one currently forced character. It must not read the hidden solution to choose the hint.

4.5 Experiments (10 pts)

Evaluate the complete system on several 3×3 and 4×4 puzzles. A 5×5 puzzle may be included as a stress case.
For each case, record:

1. Number of primary and auxiliary variables.
2. Number of CNF clauses.
3. Number of SAT calls.
4. Decisions, propagations, and backtracks.
5. Number of deduction steps or reveal waves.
6. Runtime.

Use the same puzzle set for all comparisons. Report failures and timeouts rather than removing them from the results.

4.6 Report (30 pts)

Submit a well-formatted PDF report including:

- **Project Planning and Task Distribution**:
  - Document the responsibilities of each team member, including which tasks were assigned and each member's completion percentage.
  - Example: if the group score is 9.0 and Student A has a completion percentage of 90%, then A's individual score is $9.0 \times 90\% = 8.1$.
  - It is important to evaluate contributions **fairly** and **accurately**.
- **Problem formulation**: variables, clue semantics, Knowledge Base, and the reveal protocol.
- **CNF encoding**: derivations for at least one binary clue and one counting clue.
- **Algorithms**: DPLL, entailment classification, uniqueness checking, and the deduction loop.
- **Experiments**: tables or charts, observations, failure cases, and limitations.
- **References** and **Appendix**:
  - List all references (papers, books, online tutorials, GitHub repositories, etc.).
  - Include your Appendix, for example: how you used Generative AI (LLMs) in your work, what prompts you used.

4.7 Demonstration Video (10 pts)

Create a video of approximately 5 to 10 minutes. Upload the video to **Google Drive** and set the sharing permission so that anyone with the link can view it. The video must demonstrate:

- **Graphical Interface (5 pts)**: Demonstrate the GUI of your program.
- **Feature Presentation (5 pts)**: Demonstrate the implemented functionalities:
  - Playing the game manually.
  - Running your solvers (e.g., choose a stage, run solvers, show the found solution).

The video must contain narration or subtitles in English or Vietnamese to make it easy to follow. Verify the Google Drive permission using a private/incognito browser window before submission. Do **NOT** submit a YouTube link.

5 Submission

Please follow the following submission guidelines:

- Your source code and report must be contributed in the form of a compressed file (.zip, .rar) and named according to the format `StudentID1_StudentID2_...`
- If the compressed file is larger than 20MB, upload it to Google Drive and share it via a link. **Absolutely no modifications are allowed after the deadline**.

Example details of the directory organization:

```
StudentID1_StudentID2_...
├── Source
│   ├── main.py
│   ├── README.md (how to run source code)
│   ├── requirements.txt (libraries to be installed)
│   └── ...
└── Report.pdf (included demo video URLs)
```

6 Notices

Please pay attention to the following notices:

- This is a **GROUP** assignment.
- Duration: approximately 3 weeks.
- AI tools are **not restricted**; however, students should use them wisely. Lab instructors have the right to conduct additional oral interviews with random groups to assess their knowledge of the project.
- Any form of plagiarism, dishonesty, or misconduct will result in a grade of zero for the course.

The end.